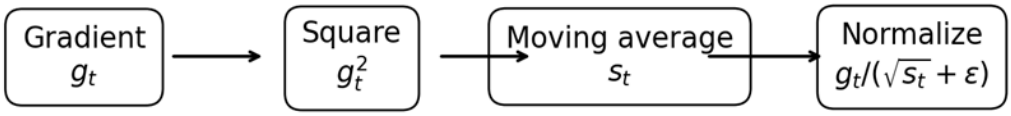
RMSprop, Adam and AdamW

RMSprop

RMSprop (Root Mean Square Propagation) controls the learning rate of each parameter using a moving average of squared gradients. It reduces overly large updates in directions where gradients are consistently large and can make training more stable.



Core equations:

$$s_t = \rho s_{t-1} + (1-\rho) g_t^2$$
$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{s_t}+\varepsilon} g_t$$

- g_t: gradient at step t.
- s_t: exponential moving average of squared gradients.
- ρ (rho): decay rate, commonly around 0.95..
- η (eta): learning rate.
- ε (epsilon): small value such as 10^{-8} to avoid division by zero.

How RMSprop works

- Compute the gradient using backpropagation.
- Square each gradient element.

- Update the moving average of squared gradients.
- Divide the gradient by the root of this moving average.
- Update the parameters.

Advantages of RMSprop

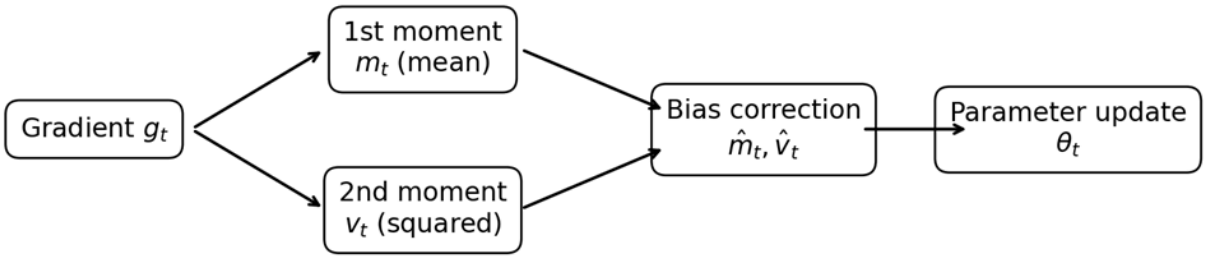
- Adapts the effective learning rate per parameter.
- Usually converges faster than basic SGD on many problems.
- Works well when gradients have different magnitudes.
- Helps stabilize optimization in noisy or non-stationary settings.

Limitations

- Requires hyperparameters such as learning rate and decay rate.
- Does not explicitly maintain a first-moment estimate of gradients.
- Its behavior can still be sensitive to learning-rate choice.

Adam

Adam (Adaptive Moment Estimation) combines ideas from momentum and RMSprop. It maintains two moving averages: the first moment (mean of gradients) and the second raw moment (mean of squared gradients).



Adam equations:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$
$$\hat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1-\beta_2^t}$$
$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon}$$

- m_t is the first-moment estimate and acts similarly to momentum.
- v_t is the second-moment estimate and controls adaptive scaling.
- β_1 is commonly 0.9; β_2 is commonly 0.999.
- Bias correction compensates for the fact that m and v start at zero.

Why bias correction is needed

At the beginning of training, m_0 and v_0 are initialized to zero. Their early moving averages are therefore biased toward zero. Dividing by $(1-\beta_1^t)$ and $(1-\beta_2^t)$ corrects this initialization effect.

Adam algorithm – step by step

- Calculate gradient g_t.
- Update first moment m_t.
- Update second moment v_t.
- Apply bias correction to obtain $\hat{m}$_t and $\hat{v}$_t.
- Compute the adaptive update.
- Update model parameters.

Advantages of Adam

- Fast convergence in many practical deep-learning tasks.
- Combines momentum-like direction tracking with adaptive step sizes.
- Usually requires less manual learning-rate tuning than plain SGD.
- Useful for sparse and noisy gradients.

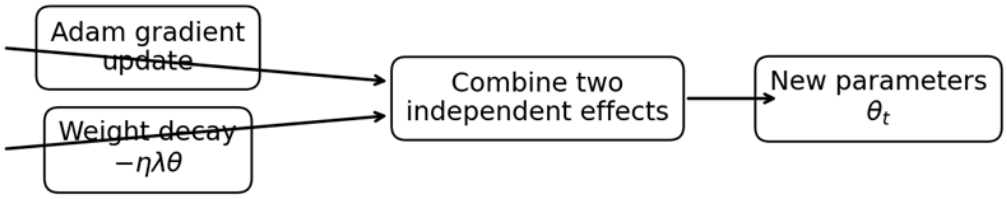
Limitations of Adam

- Can generalize differently from SGD on some tasks.

- Uses additional memory for first- and second-moment states.
- Hyperparameters can still matter, especially learning rate.

AdamW

AdamW is Adam with decoupled weight decay. The key idea is to apply weight decay directly to the parameters rather than treating it as part of the gradient used by Adam's adaptive moments.



AdamW update:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} - \eta\lambda\theta_{t-1}$$

The first term is Adam's adaptive gradient update. The second term directly shrinks the parameters. λ (lambda) controls the strength of weight decay.

Adam vs AdamW

Adam: gradient update + L2 term coupled in the gradient
AdamW: gradient update and weight decay are decoupled

- Adam with L2 regularization couples the regularization term to the gradient transformation.
- AdamW decouples weight decay from the adaptive gradient update.
- This makes the weight-decay hyperparameter behave more directly as a parameter-shrinkage control.

Why AdamW is popular

- Often gives strong optimization and regularization together.
- Commonly used for transformer and modern deep-learning models.
- Makes weight decay easier to tune independently from Adam's adaptive update.

Comparison

Optimizer	Main idea	Momentum?	Adaptive scaling?	Weight decay
RMSprop	Moving average of squared gradients	No explicit first moment	Yes	Not built in
Adam	First + second moments	Yes	Yes	Classical Adam may couple L2
AdamW	Adam + decoupled weight decay	Yes	Yes	Yes, decoupled

Intuitive Example

Imagine a hiker descending a mountain. SGD uses the current slope. RMSprop also considers how large the recent slopes have been, so it reduces the step size in directions with consistently large gradients. Adam additionally remembers the general direction of recent slopes through its first-moment estimate. AdamW performs Adam's adaptive movement while separately applying a small shrinkage to the parameters, discouraging excessively large weights.

Common Hyperparameters

Hyperparameter	RMSprop	Adam	AdamW
Learning rate η	Often ~0.001	Often ~0.001	Often ~0.001; model/task dependent
Decay / β_1	ρ ~0.95	β_1 ~0.95	β_1 ~0.95
β_2	—	~0.999	~0.999
Weight decay λ	—	Task-dependent / implementation-dependent	Commonly tuned explicitly

When to Use Which?

- Use RMSprop when you want a simple adaptive optimizer and the problem benefits from per-parameter scaling.
- Use Adam as a strong general-purpose baseline, especially for fast experimentation.
- Use AdamW when using Adam-like optimization and you want explicit, decoupled weight decay; it is a common choice for modern deep-learning models.

Key Takeaways

- RMSprop adapts learning rates using a moving average of squared gradients.
- Adam combines a momentum-like first moment with RMSprop-like second-moment scaling.
- Adam uses bias correction because its moment estimates start at zero.
- AdamW separates weight decay from Adam's adaptive gradient update.